

The background is an abstract geometric pattern composed of numerous overlapping triangles. The color gradient transitions from a bright yellow on the left side, through orange and red, to a deep blue on the right side. The triangles vary in size and orientation, creating a complex, crystalline texture.

Lec3-2 Lexical and syntax analyzer

CO523PL 23-12-2025

Lexical Analysis and Syntax Analysis are two fundamental phases of a **compiler** (or interpreter).

1. Lexical Analysis (Scanner)

What it does

- Lexical analysis is the **first phase** of a compiler.
- It reads the source code **character by character** and groups them into meaningful units called **tokens**.

Input & Output

Input: Source code (characters)

Output: Tokens

Tokens can be:

- Keywords (if, while, int)
- Identifiers (x, total)
- Operators (+, =, *)
- Literals (10, "hello")
- Symbols (;, {, })

Example

Source code:

```
int sum = 10;
```

* Tokens produced:

int → keyword

sum → identifier

= → operator

10 → literal

; → symbol

Responsibilities

- Remove **whitespace and comments**
- Detect **lexical errors** (invalid characters, e.g. @ in C)
- Convert characters into tokens

2. Syntax Analysis (Parser)

What it does

- Syntax analysis is the **second phase** of a compiler.
It checks whether the sequence of tokens follows the **grammar rules** of the programming language.

Input & Output

- **Input:** Tokens from lexical analyzer
- **Output:** Parse tree / Syntax tree (or error messages)

Example

- Tokens:

`int sum = 10 ;`

The parser checks:

- Is the declaration format correct?
- Does it follow the language grammar?

If correct → builds a **parse tree**

If incorrect → reports a **syntax error**

Example of Syntax Error

- `int = sum 10;`

✗ Error: invalid variable declaration order

Responsibilities

- Check **statement structure**
- Ensure correct **order of tokens**
- Detect **syntax errors** (missing `;`, wrong brackets, etc.)

Feature	Lexical Analysis	Syntax Analysis
Phase	First	Second
Works on	Characters	Tokens
Output	Tokens	Parse Tree
Errors	Invalid symbols	Grammar errors
Tool	Scanner	Parser

Simple Analogy

- Lexical Analysis:** Breaking a sentence into words
- Syntax Analysis:** Checking if the sentence is grammatically correct

HOW COMPILERS WORK PART 2

LEXICAL ANALYSIS

Lexical analysis is the first phase of a compiler. It takes modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code. If the lexical analyzer finds a token invalid, it generates an error. The lexical analyzer works closely with the syntax analyzer. It reads character streams from the source code, checks for legal tokens, and passes the data to the syntax analyzer when it demand

TOKENS

Lexemes are said to be a sequence of characters (alphanumeric) in a token. There are some predefined rules for every lexeme to be identified as a valid token. These rules are defined by grammar rules, by means of a pattern. A pattern explains what can be a token, and these patterns are defined by means of regular expressions.

In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

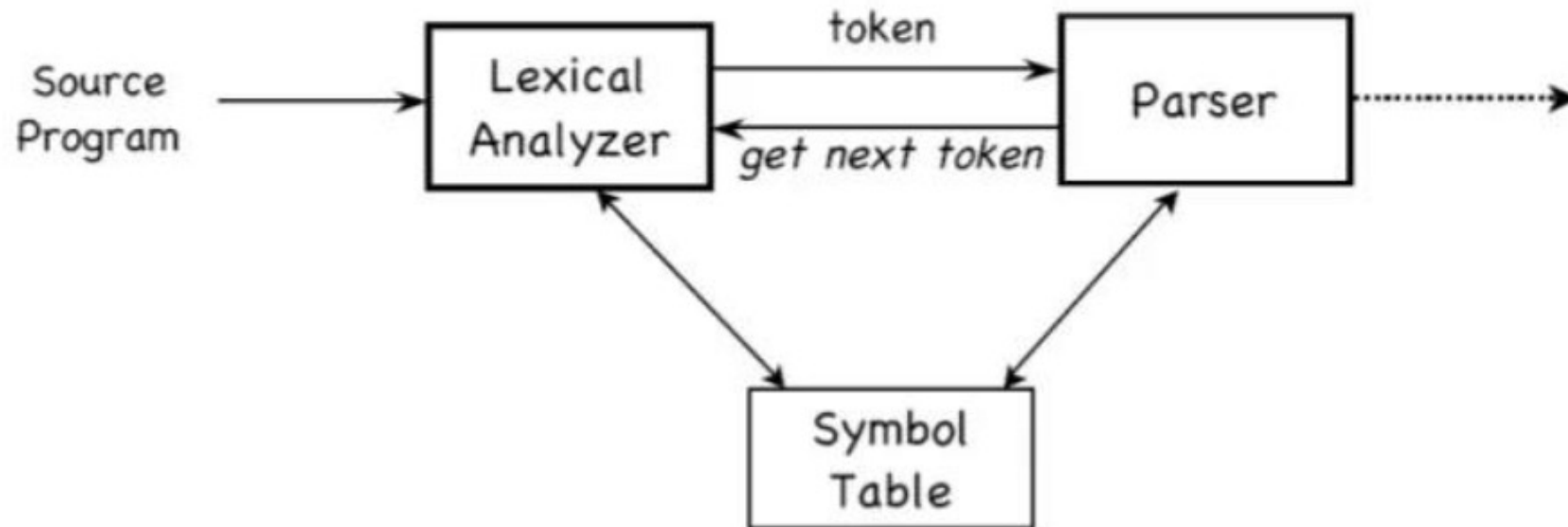
PARSER

A parser is a **compiler or interpreter component that breaks data into smaller elements for easy translation into another language**. A parser takes input in the form of a sequence of tokens, interactive commands, or program instructions and breaks them up into parts that can be used by other components in programming

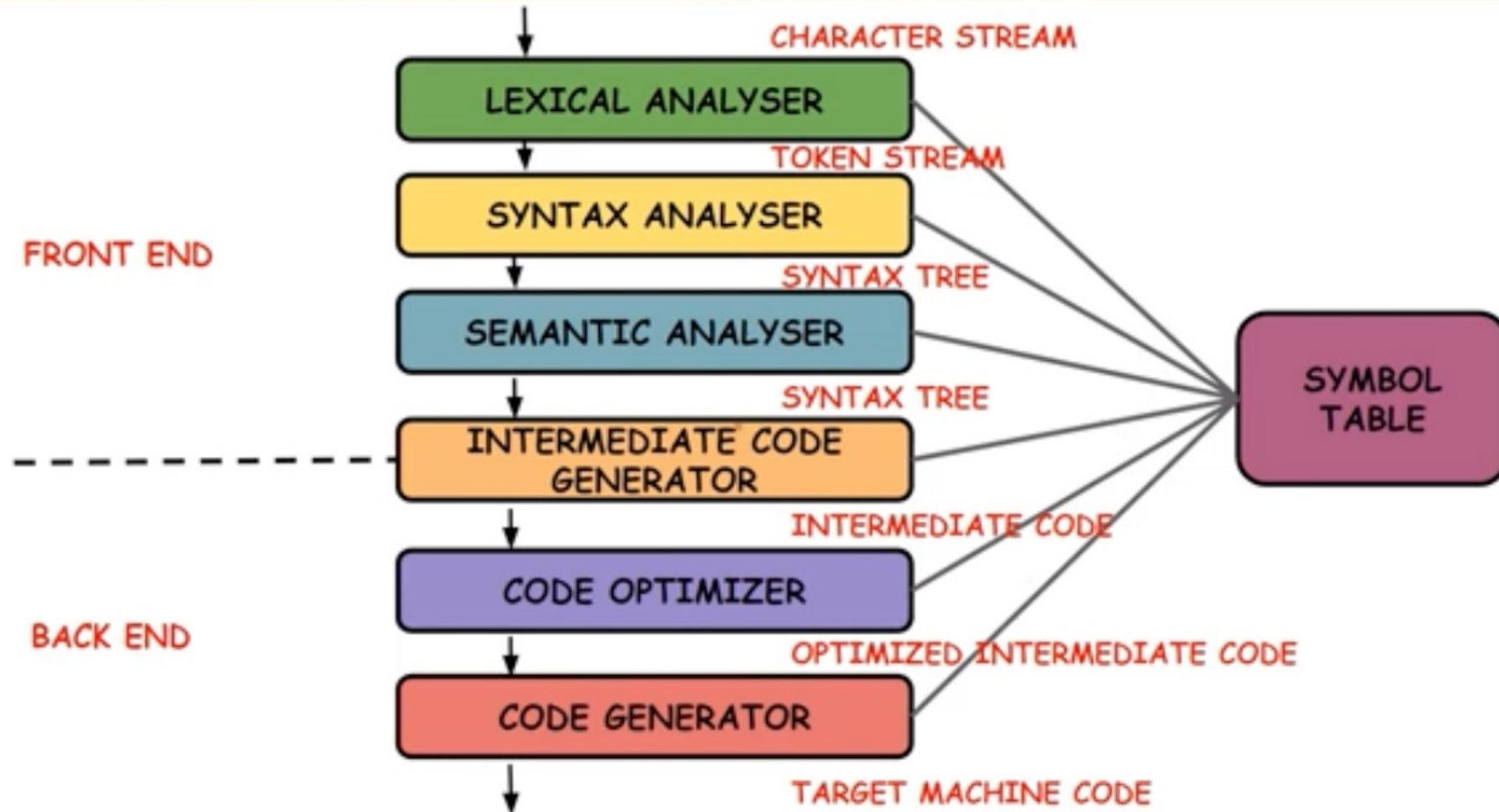
SYMBOL TABLE

Symbol Table is **an important data structure created and maintained by the compiler in order to keep track of semantics of variables** i.e. it stores information about the scope and binding information about names, information about instances of various entities such as variable and function names, classes, objects, etc.

Lexical Analysis

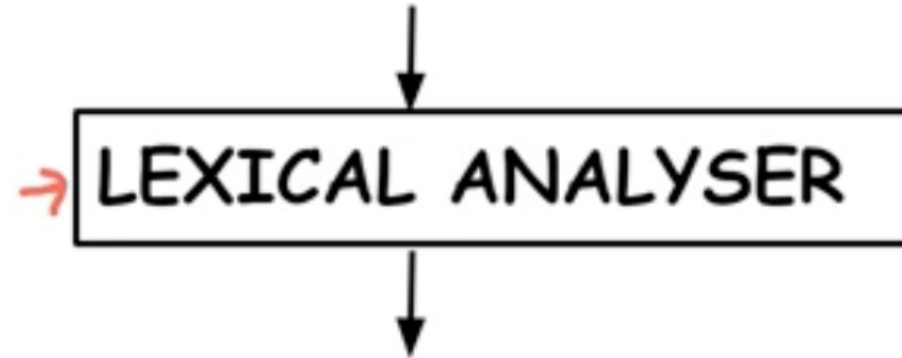


PHASES OF COMPILER



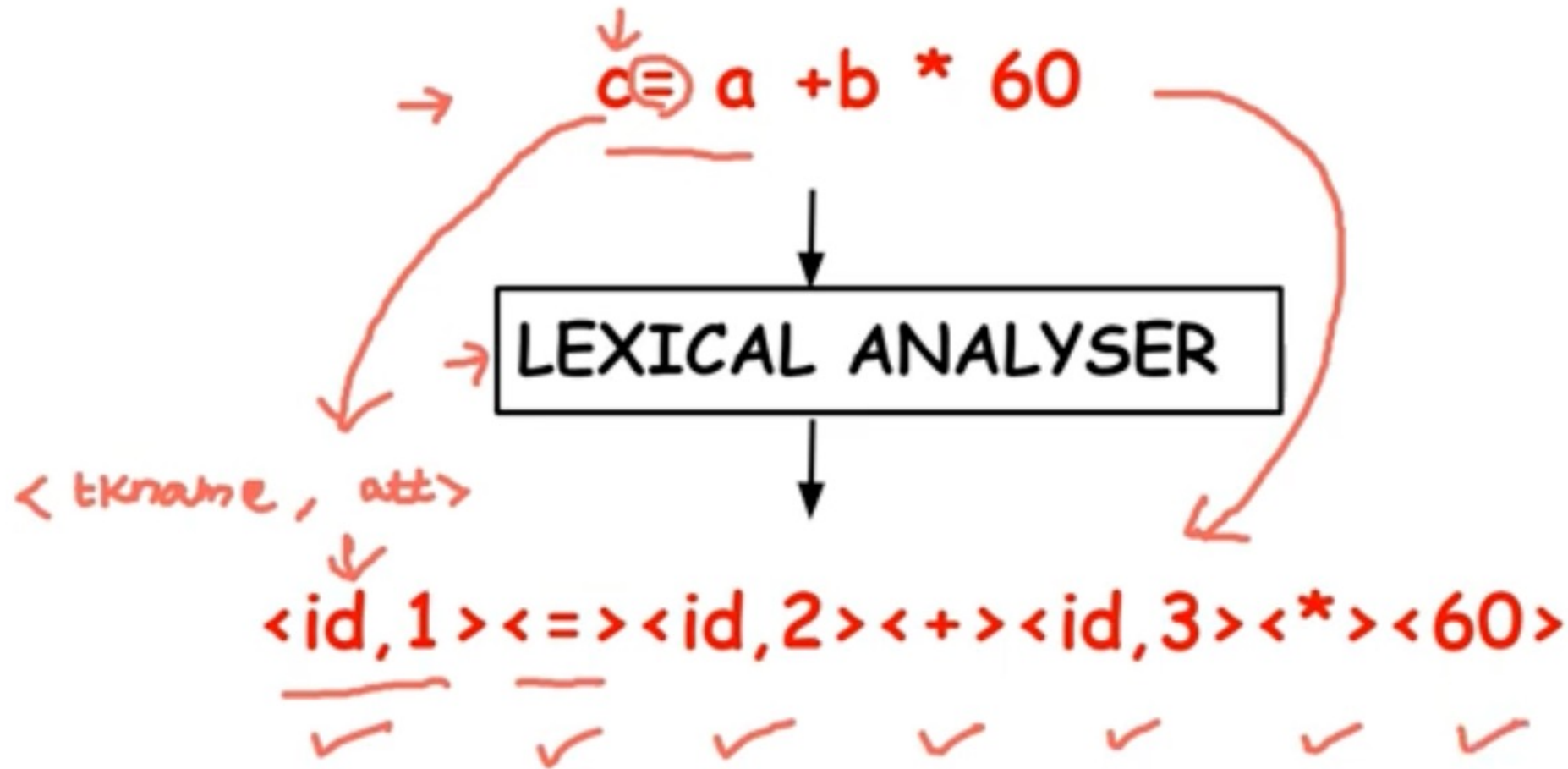
PHASES OF COMPILER - LEXICAL ANALYSER

→ c = a + b * 60



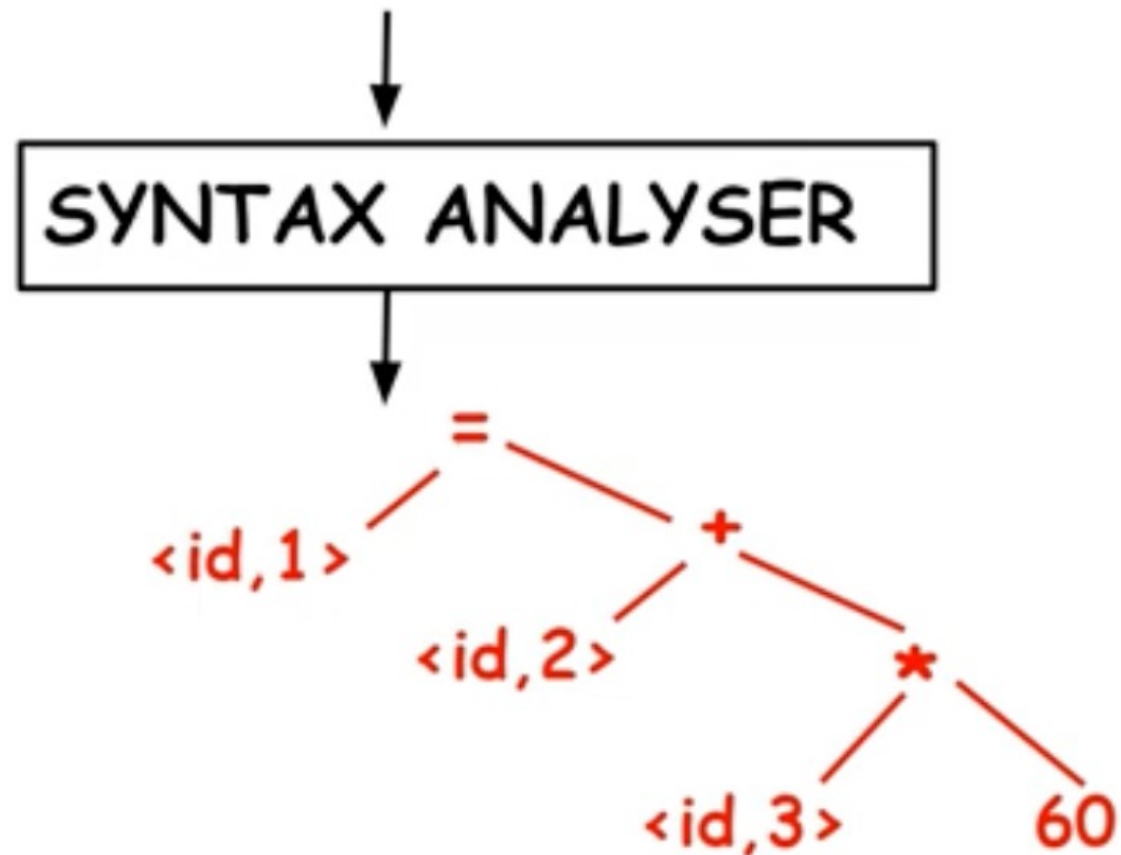
<id,1><=><id,2><+><id,3><*><60>

PHASES OF COMPILER - LEXICAL ANALYSER



PHASES OF COMPILER - SYNTAX ANALYSER

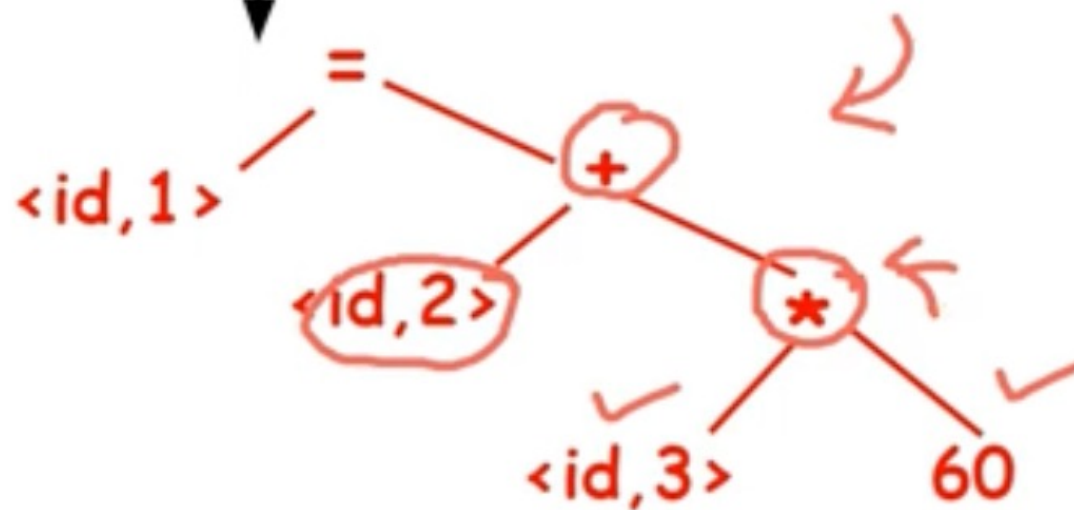
→ $\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$



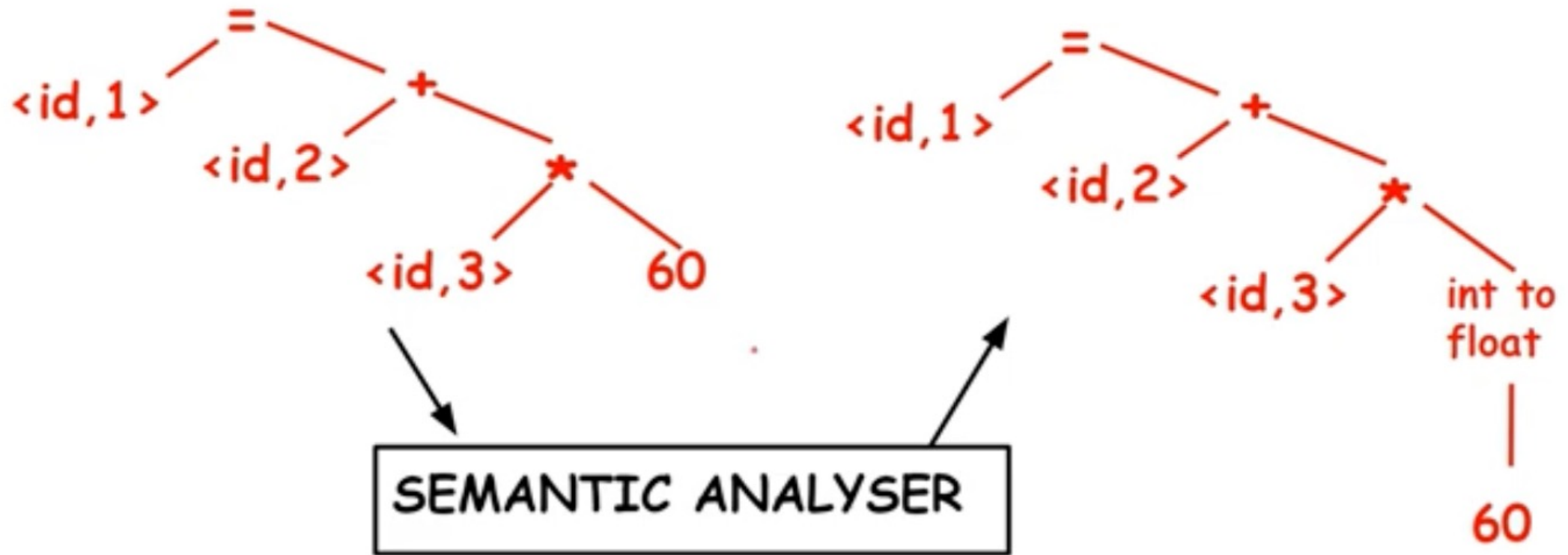
PHASES OF COMPILER - SYNTAX ANALYSER

→ $\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

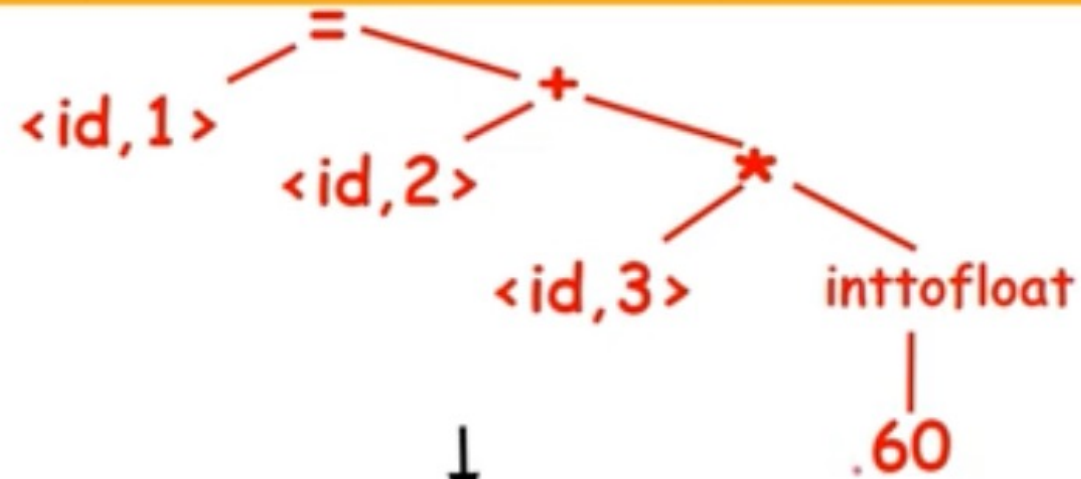
SYNTAX ANALYSER



PHASES OF COMPILER - SEMANTIC ANALYSER



PHASES OF COMPILER - INTERMEDIATE CODE GENERATOR



INTERMEDIATE CODE GENERATOR

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 * t2
id1 = t3
```

PHASES OF COMPILER - CODE OPTIMIZER

t1 = inttofloat(60)

t2 = id3 * t1

t3 = id2 + t2

id1 = t3



CODE OPTIMIZER



t1 = id3 * 60.0

id1 = id2 + t1

PHASES OF COMPILER - CODE OPTIMIZER

```
{  
  t1 = inttofloat(60)  
  t2 = id3 * t1  
  t3 = id2 + t2  
  id1 = t3  
}
```

CODE OPTIMIZER

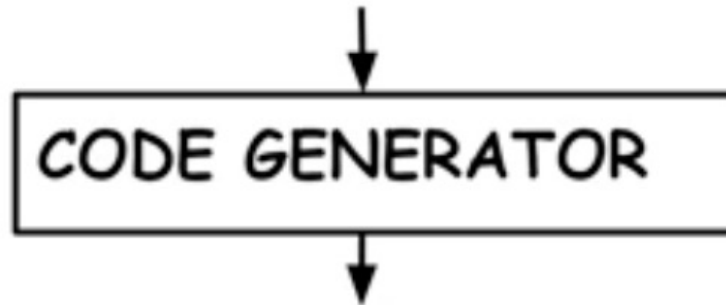
```
t1 = id3 * 60.0 ✓
```

```
id1 = id2 + t1 ✓
```


PHASES OF COMPILER - CODE GENERATOR

$t1 = id3 * 60.0$

$id1 = id2 + t1$



LDF R2,id3

MULF R2,R2,#60.0

LDF R1,id2

ADDF R1,R1,R2

STF id1,R1

PHASES OF COMPILER - CODE GENERATOR

$t1 = id3 * 60.0$

$id1 = id2 + t1$

CODE GENERATOR

LDF R2,id3

MULF R2,R2,#60.0

LDF R1,id2

ADDF R1,R1,R2

STF id1,R1

PHASES OF COMPILER - CODE GENERATOR

$t1 = id3 * 60.0$

$id1 = id2 + t1$

CODE GENERATOR

→ LDF R2, id3

MULF R2, R2, #60.0

LDF R1, id2

ADDF R1, R1, R2

STF id1, R1

PHASES OF COMPILER - CODE GENERATOR

$t1 = id3 * 60.0$

$id1 = id2 + t1$

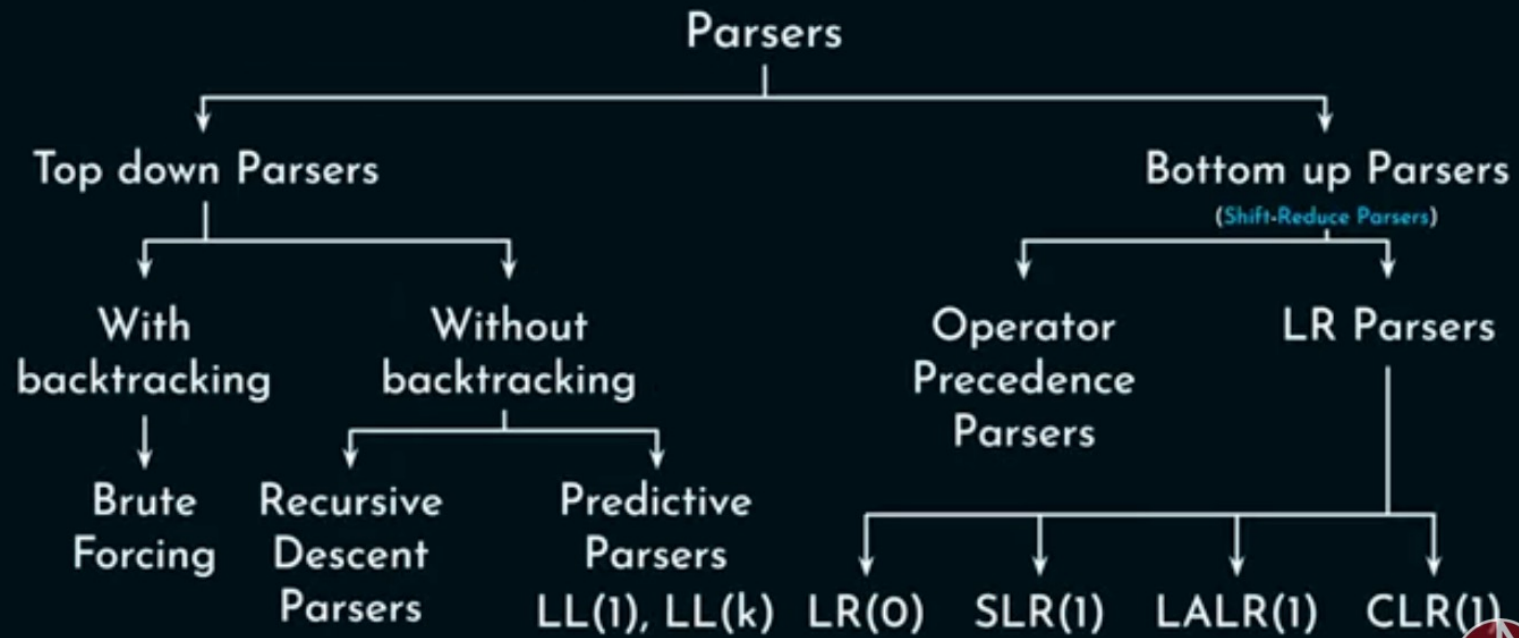
CODE GENERATOR

→ LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1

Compiler Design

- Topdown parser LL(1) Parsers

Classification of Parsers:



Recursive Descent Parser:

$E \rightarrow iE'$

$E' \rightarrow +iE' \mid \varepsilon$

```
1.  E()
2.  {
3.      if(look_ahead=='i')
4.      {
5.          match('i');
6.          E'();
7.      }
8.  }
```

```
1.  match(char c)
2.  {
3.      if(look_ahead==c)
4.          look_ahead = getchar();
5.      else
6.          printf("ERROR!");
7.  }
```

```
1.  E'()
2.  {
3.      if(look_ahead=='+')
4.      {
5.          match('+');
6.          match('i');
7.          E'();
8.      }
9.      else
10.         return;
```

```
1.  main()
2.  {
3.      E();
4.      if(look_ahead=='$')
5.          printf("Parsing Successful!");
6.  }
```

Input: i + i \$



Recursive Descent Parser:

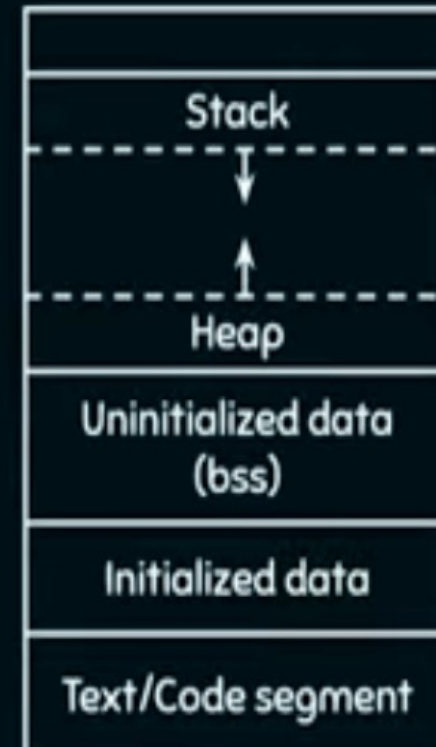
```
1. f()
2. {
3.     if(look_ahead=='i')
4.     {
5.         match('i');
6.         f();
7.     }
8. }

1. E'()
2. {
3.     if(look_ahead=='i')
4.     {
5.         match('i');
6.         match('+');
7.         f();
8.     }
9.     else
10.    return;

1. match(char c)
2. {
3.     if(look_ahead==c)
4.         look_ahead = getch();
5.     else
6.         printf("ERROR");
7. }
```

$E \rightarrow iE'$

$E' \rightarrow +iE' \mid \epsilon$



Input: $i + i \$$

Recursive Descent Parser:

$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \varepsilon$

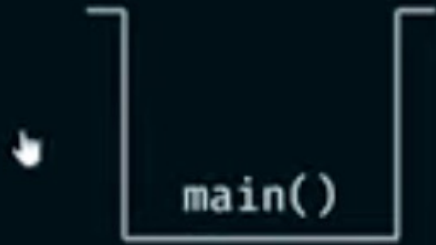
```
1.  E()  
2.  {  
3.      if(look_ahead=='i')  
4.      {  
5.          match('i');  
6.          E'();  
7.      }  
8.  }
```

```
1.  E'()  
2.  {  
3.      if(look_ahead=='+')  
4.      {  
5.          match('+');  
6.          match('i');  
7.          E'();  
8.      }  
9.      else  
10.         return;
```

```
1.  match(char c)  
2.  {  
3.      if(look_ahead==c)  
4.          look_ahead = getchar();  
5.      else  
6.          printf("ERROR!");  
7.  }
```

```
1.  main()  
2.  {  
3.      E();  
4.      if(look_ahead=='$')  
5.          printf("Parsing Successful!");  
6.  }
```

Input: i + i \$



main()

Recursive Descent Parser:

$E \rightarrow iE'$

$E' \rightarrow +iE' \mid \varepsilon$

```
1. E()
2. {
3.     if(look_ahead=='i')
4.     {
5.         match('i');
6.         E'();
7.     }
8. }
```

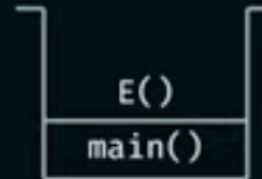
```
1. match(char c)
2. {
3.     if(look_ahead==c)
4.         look_ahead = getchar();
5.     else
6.         printf("ERROR!");
7. }
```

```
1. E'()
2. {
3.     if(look_ahead=='+')
4.     {
5.         match('+');
6.         match('i');
7.         E'();
8.     }
9.     else
10.    return;
```

```
1. main()
2. {
3.     E();
4.     if(look_ahead=='$')
5.         printf("Parsing Successful!");
6. }
```

E

Input: i + i \$



Recursive Descent Parser:

$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \epsilon$

```
1. E()  
2. {  
3.     if(look_ahead=='i')  
4.     {  
5.         match('i');  
6.         E'();  
7.     }  
8. }
```

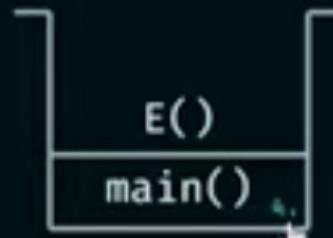
```
1. match(char c)  
2. {  
3.     if(look_ahead==c)  
4.         look_ahead = getchar();  
5.     else  
6.         printf("ERROR!");  
7. }
```

E

```
1. E'()  
2. {  
3.     if(look_ahead=='+')  
4.     {  
5.         match('+');  
6.         match('i');  
7.         E'();  
8.     }  
9.     else  
10.        return;
```

```
1. main()  
2. {  
3.     E();  
4.     if(look_ahead=='$')  
5.         printf("Parsing Successful!");  
6. }
```

Input: i + i \$



Recursive Descent Parser:

$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \varepsilon$

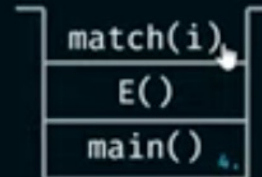
```
1. E()  
2. {  
3.   if(look_ahead=='i')  
4.   {  
5.       match('i');  
6.       E'();  
7.   }  
8. }
```

```
1. E'()  
2. {  
3.   if(look_ahead=='+')  
4.   {  
5.       match('+');  
6.       match('i');  
7.       E'();  
8.   }  
9.   else  
10.  return;
```

```
1. match(char c)  
2. {  
3.   if(look_ahead==c)  
4.       look_ahead = getchar();  
5.   else  
6.       printf("ERROR!");  
7. }
```

```
1. main()  
2. {  
3.   E();  
4.   if(look_ahead=='$')  
5.       printf("Parsing Successful!");  
6. }
```

Input: i + i \$



Recursive Descent Parser:

$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \varepsilon$

```
1. E()  
2. {  
3.     if(look_ahead=='i')  
4.     {  
5.         match('i');  
6.         E';  
7.     }  
8. }
```

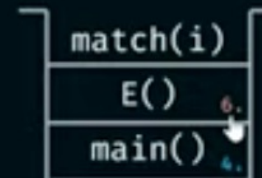
```
1. match(char c)  
2. {  
3.     if(look_ahead==c)  
4.         look_ahead = getchar();  
5.     else  
6.         printf("ERROR!");  
7. }
```

E
|
i

```
1. E'()  
2. {  
3.     if(look_ahead=='+')  
4.     {  
5.         match('+');  
6.         match('i');  
7.         E';  
8.     }  
9.     else  
10.    return;
```

```
1. main()  
2. {  
3.     E();  
4.     if(look_ahead=='$')  
5.         printf("Parsing Successful!");  
6. }
```

Input: i + i \$



Recursive Descent Parser:

$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \varepsilon$

```
1. E()  
2. {  
3.     if(look_ahead=='i')  
4.     {  
5.         match('i');  
6.         E'();  
7.     }  
8. }
```

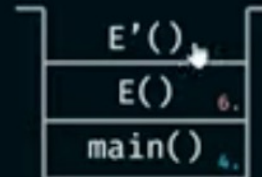
```
1. match(char c)  
2. {  
3.     if(look_ahead==c)  
4.         look_ahead = getchar();  
5.     else  
6.         printf("ERROR!");  
7. }
```

E
/
i

```
1. E'()  
2. {  
3.     if(look_ahead=='+')  
4.     {  
5.         match('+');  
6.         match('i');  
7.         E'();  
8.     }  
9.     else  
10.    return;
```

```
1. main()  
2. {  
3.     E();  
4.     if(look_ahead=='$')  
5.         printf("Parsing Successful!");  
6. }
```

Input: i + i \$



Recursive Descent Parser:

```
1. E()
2. {
3.     if(look_ahead=='i')
4.     {
5.         match('i');
6.         E'();
7.     }
8. }
```

```
1. E'()
2. {
3.     if(look_ahead=='+')
4.     {
5.         match('+');
6.         match('i');
7.         E'();
8.     }
9.     else
10.    return;
```

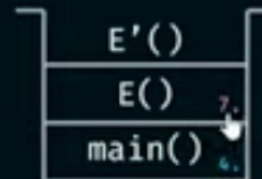
```
1. match(char c)
2. {
3.     if(look_ahead==c)
4.         look_ahead = getchar();
5.     else
6.         printf("ERROR!");
7. }
```

```
1. main()
2. {
3.     E();
4.     if(look_ahead=='$')
5.         printf("Parsing Successful!");
6. }
```

$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \varepsilon$



Input: i + i \$



Recursive Descent Parser:

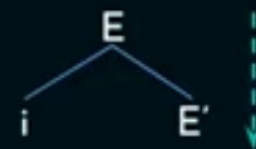
```
1. E()
2. {
3.     if(look_ahead=='i')
4.     {
5.         match('i');
6.         E'();
7.     }
8. }
```

```
1. E'()
2. {
3.     if(look_ahead=='+')
4.     {
5.         match('+');
6.         match('i');
7.         E'();
8.     }
9.     else
10.    return;
```

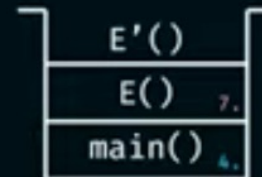
```
1. match(char c)
2. {
3.     if(look_ahead==c)
4.         look_ahead = getchar();
5.     else
6.         printf("ERROR!");
7. }
```

```
1. main()
2. {
3.     E();
4.     if(look_ahead=='$')
5.         printf("Parsing Successful!");
6. }
```

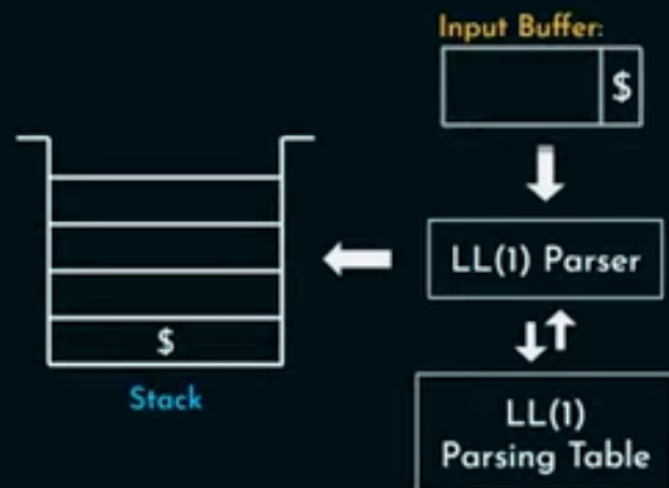
$E \rightarrow iE'$
 $E' \rightarrow +iE' \mid \epsilon$



Input: i + i \$

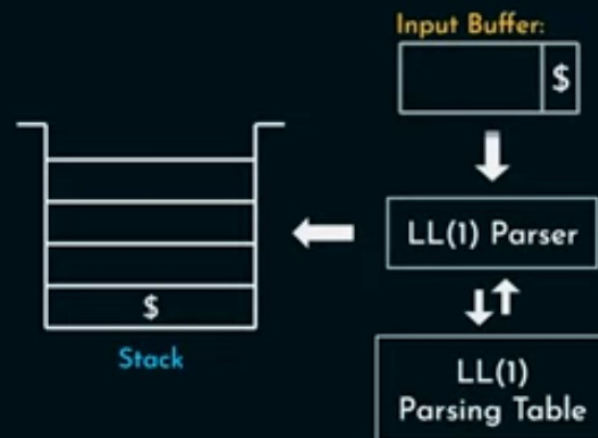


LL(1) Parser:

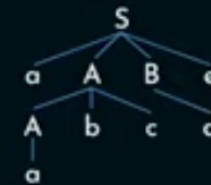


LL(1) Parser:

Scan from Left to right.



Top down approach:



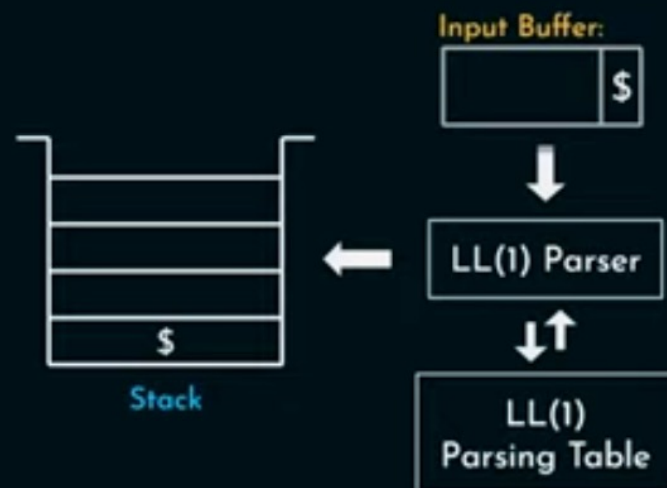
$S \Rightarrow aABe$
 $\Rightarrow aAbcBe$
 $\Rightarrow aabcBe$
 $\Rightarrow aabcde$

Decision:
Which production
to use.

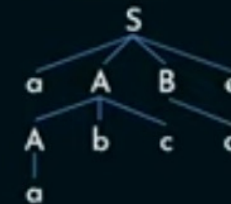
(Left most Derivation)

LL(1) Parser:

- “1” look-ahead symbol.
- Use **Left most derivation**.
- Scan from **Left to right**.



Top down approach:



$S \Rightarrow aABe$
 $\Rightarrow aAbcBe$
 $\Rightarrow aabcBe$
 $\Rightarrow aabcde$

(Left most Derivation)

Decision:

Which production to use.

LL(1) Parser:

1. FIRST():

Given any non-terminal of a CFG, if we derive all the possible strings from it, the **first terminal(s)** is the FIRST() of the non-terminal.

e.g.(1): $S \rightarrow aABC$ **FIRST(S):**

$A \rightarrow b$

$B \rightarrow c$

$C \rightarrow d$



(1): $S \rightarrow aABC$

$A \rightarrow b$

$B \rightarrow c$

$C \rightarrow d$

FIRST(S):



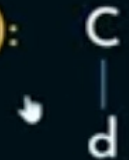
FIRST(B):



FIRST(A):



FIRST(C):



e.g.(2): $S \rightarrow ABC$ $\text{FIRST}(S): \{ a, b \}$

$A \rightarrow a \mid \varepsilon$

$B \rightarrow b$

$C \rightarrow c$

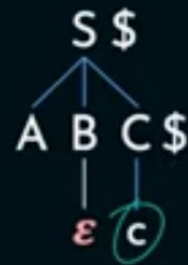


LL(1) Parser:

2. FOLLOW():

During the process of derivation, the **terminal(s)** that could follow the non-terminal are to be considered as FOLLOW() of the non-terminal.

e.g.: $S \rightarrow ABC$
 $A \rightarrow a$
 $B \rightarrow b \mid \epsilon$
 $C \rightarrow c$



FOLLOW(S): { \$ }

FOLLOW(A): { b, c }

FOLLOW(B): { c }

FOLLOW(C): { \$ }

Compiler Design

First & Follow

Derivation of FIRST:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow \text{id} \mid (E)$

$\text{FIRST}(F) = \{\text{id}, (\}$

$\text{FIRST}(T') = \{*, \varepsilon\}$

$\text{FIRST}(T) = \{\text{id}, (\}$

$\text{FIRST}(E') = \{+, \varepsilon\}$

$\text{FIRST}(E) = \text{FIRST}(T) = \{\text{id}, (\}$

Derivation of FIRST:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow id \mid (E)$

$FIRST(F) = \{id, (\}$

$FIRST(T') = \{*, \varepsilon\}$

$FIRST(T) = \{id, (\}$

$FIRST(E') = \{+, \varepsilon\}$

$FIRST(E) = \{id, (\}$

	FIRST
$E \rightarrow TE'$	$\{id, (\}$
$E' \rightarrow +TE' \mid \varepsilon$	$\{+, \varepsilon\}$
$T \rightarrow FT'$	$\{id, (\}$
$T' \rightarrow *FT' \mid \varepsilon$	$\{*, \varepsilon\}$
$F \rightarrow id \mid (E)$	$\{id, (\}$



Derivation of FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	$\{id, (\}$	$\{ \$,) \}$
$E' \rightarrow +TE' \mid \varepsilon$	$\{ +, \varepsilon \}$	$\{ \$,) \}$
$T \rightarrow FT'$	$\{id, (\}$	$\{ + \}$
$T' \rightarrow *FT' \mid \varepsilon$	$\{ *, \varepsilon \}$	
$F \rightarrow id \mid (E)$	$\{id, (\}$	

$$FOLLOW(T) = FIRST(E') = \{ +, \varepsilon \}$$

1. The following terminal symbol will be selected as FOLLOW.

Derivation of FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{\$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,\varepsilon}	{\$,)}
$T \rightarrow FT'$	{id,(}	{+}
$T' \rightarrow *FT' \mid \varepsilon$	{*,\varepsilon}	
$F \rightarrow id \mid (E)$	{id,(}	

$$\text{FOLLOW}(T) = \text{FIRST}(E') = \{ +, \varepsilon \}$$

1. The following terminal symbol will be selected as FOLLOW.
2. The FIRST of the following non-terminal will be selected as FOLLOW.

Derivation of FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{\$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,\varepsilon}	{\$,)}
$T \rightarrow FT'$	{id,(}	{+,\$,)} ↓
$T' \rightarrow *FT' \mid \varepsilon$	{*,\varepsilon}	
$F \rightarrow id \mid (E)$	{id,(}	

$$\text{FOLLOW}(T) = \text{FIRST}(E') = \{ +, \varepsilon \}$$

1. The following terminal symbol will be selected as FOLLOW.
2. The FIRST of the following non-terminal will be selected as FOLLOW.
3. If it is the right most in the RHS, the FOLLOW of the LHS will be selected.

Derivation of FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	$\{id, (\}$	$\{ \$,) \}$
$E' \rightarrow +TE' \mid \varepsilon$	$\{ +, \varepsilon \}$	$\{ \$,) \}$
$T \rightarrow FT'$	$\{id, (\}$	$\{ +, \$,) \}$
$T' \rightarrow *FT' \mid \varepsilon$	$\{ *, \varepsilon \}$	$\{ +, \$,) \}$
$F \rightarrow id \mid (E)$	$\{id, (\}$	$\{ *, +, \$,) \}$

$$\begin{aligned}\text{FOLLOW}(F) &= \text{FIRST}(T') = \{ *, \varepsilon \} \\ &= \{ * \} \cup \text{FOLLOW}(T) \cup \text{FOLLOW}(T') \\ &= \{ *, +, \$,) \}\end{aligned}$$

Derivation of FIRST & FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{\$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+, ε }	{\$,)}
$T \rightarrow FT'$	{id,(}	{+,\$,)}
$T' \rightarrow *FT' \mid \varepsilon$	{*, ε }	{+,\$,)}
⤵ $F \rightarrow id \mid (E)$	{id,(}	{*,+,\$,)}

Context Free Grammar:

1. $E \rightarrow E + E$
2. $E \rightarrow E \times E$
3. $E \rightarrow id$

id + id x id

Left Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

Right Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow E + E \times E$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

Parse Tree Derivation



Context Free Grammar:

$E \rightarrow E + E \mid E \times E \mid id$

id + id x id

Left Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

$E \Rightarrow E \times E$
 $\Rightarrow E + E \times E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

Right Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow E + E \times E$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

Parse Tree Derivation:



Context Free Grammar:

$E \rightarrow E + E \mid E \times E \mid id$

id + id x id

Left Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

$E \Rightarrow E \times E$
 $\Rightarrow E + E \times E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

Right Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow E + E \times E$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

$E \Rightarrow E \times E$
 $\Rightarrow E \times id$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

Parse Tree Derivation



Context Free Grammar:

$E \rightarrow E + E \mid E \times E \mid id$

id + id x id

Left Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

$E \Rightarrow E \times E$
 $\Rightarrow E + E \times E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

Right Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow E + E \times E$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

$E \Rightarrow E \times E$
 $\Rightarrow E \times id$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

Parse Tree Derivation



Context Free Grammar:

$E \rightarrow E + E \mid E \times E \mid id$

id + id x id

Left Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

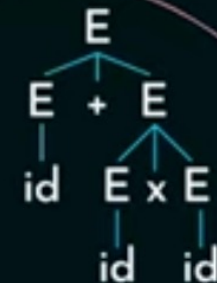
$E \Rightarrow E \times E$
 $\Rightarrow E + E \times E$
 $\Rightarrow id + E \times E$
 $\Rightarrow id + id \times E$
 $\Rightarrow id + id \times id$

Right Most Derivation:

$E \Rightarrow E + E$
 $\Rightarrow E + E \times E$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

$E \Rightarrow E \times E$
 $\Rightarrow E \times id$
 $\Rightarrow E + E \times id$
 $\Rightarrow E + id \times id$
 $\Rightarrow id + id \times id$

Parse Tree Derivation:



Compiler Designe

Ambiguity in CFGs- Solved Problems(set 1)

Q1: Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S) \mid SS \mid \varepsilon$$

UGC-NET
DEC 2018

Q1: Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S) \mid SS \mid \epsilon$$

UGC-NET
DEC 2018

Sol. Parse Tree Derivation: $()()()$



Q2: Consider the following two Grammars,

G1: $S \rightarrow SbS \mid a$

G2: $S \rightarrow aB \mid ab, A \rightarrow AB \mid a, B \rightarrow ABb \mid b$

UGC-NET
JUNE 2018

Which of the following option is correct?

- (A) Only G1 is ambiguous
- (B) Only G2 is ambiguous
- (C) Both G1 and G2 are ambiguous
- (D) Both G1 and G2 are unambiguous

Q2: Consider the following two Grammars,

G1: $S \rightarrow SbS \mid a$

G2: $S \rightarrow aB \mid ab$, $A \rightarrow AB \mid a$, $B \rightarrow ABb \mid b$

Which of the following option is correct?

Sol. G1: $S \rightarrow SbS \mid a$

ababa



Q2: Consider the following two Grammars,

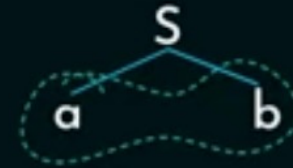
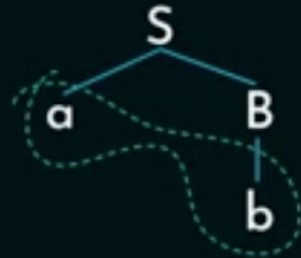
G1: $S \rightarrow SbS \mid a$

G2: $S \rightarrow aB \mid ab, A \rightarrow AB \mid a, B \rightarrow ABb \mid b$

Which of the following option is correct?

Sol. G2: $S \rightarrow aB \mid ab, A \rightarrow AB \mid a, B \rightarrow ABb \mid b$

ab



Q2: Consider the following two Grammars,

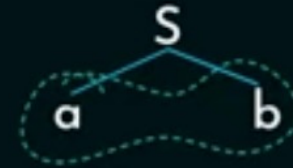
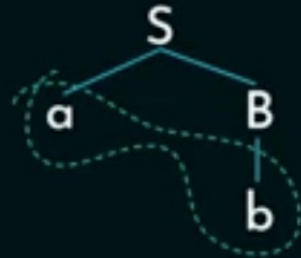
G1: $S \rightarrow SbS \mid a$

G2: $S \rightarrow aB \mid ab, A \rightarrow AB \mid a, B \rightarrow ABb \mid b$

Which of the following option is correct?

Sol. G2: $S \rightarrow aB \mid ab, A \rightarrow AB \mid a, B \rightarrow ABb \mid b$

ab



Thank you!